

03 — Informe de Pruebas Unitarias

Proyecto: Plataforma Libro de Clases Digital
Asignatura: DSY1106 — Desarrollo Fullstack III
Evaluación: Parcial N°3 — Encargo
Equipo: Cristian Monsalve / Héctor Olivares
Fecha de ejecución: 2026-06-20

1. Resumen ejecutivo

Se implementaron **pruebas unitarias y de capa web** en los cuatro módulos backend Java, utilizando **JUnit 5**, **Mockito** y **Spring Boot Test**. La cobertura de código se mide con **JaCoCo 0.8.12**, integrado en el ciclo `mvn test` de cada microservicio.

Servicio	Clases de test	Métodos de test	Cobertura instrucciones	Cobertura ramas
authService	7	33	91 %	71 %
academicService	27	147	71 %	49 %
attendanceService	13	84	80 %	60 %
apiGetaway	1	1	N/A*	N/A*
frontend-react	10	53	6 % global**	3 % global**
Total proyecto	58	318	—	—

* `apiGetaway` solo contiene configuración de rutas; el test `contextLoads` verifica el arranque del contexto Spring. JaCoCo no reporta clases instrumentables en este módulo.

** Cobertura global del frontend (incluye pantallas CRUD sin tests E2E). Los **módulos críticos probados** alcanzan cobertura alta: `permissions.ts` 94 %, `Login.tsx` 92 %, `client.ts` 100 %, validadores 80–97 %.

Resultado global: `BUILD SUCCESS` (backend) + **53/53 tests OK** (frontend) — **318 tests, 0 fallos**.

2. Herramientas y configuración

Herramienta	Versión	Uso
JUnit Jupiter	5.x (vía Spring Boot)	Framework de pruebas
Mockito	5.x	Mocks de repositorios y dependencias

Herramienta	Versión	Uso
Spring Boot Test	4.1.0	@WebMvcTest , @ExtendWith , contexto Spring
JaCoCo Maven Plugin	0.8.12	Cobertura de código
Maven Surefire	3.x	Ejecución de tests en mvn test
Vitest	4.x	Tests unitarios frontend
React Testing Library	16.x	Tests de componentes React
@vitest/coverage-v8	4.x	Cobertura de código frontend

Configuración JaCoCo en pom.xml (ejemplo authService):

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

3. Cómo ejecutar las pruebas y generar reportes

3.1 Por microservicio

```
cd authService
mvn clean test jacoco:report
# Reporte HTML: target/site/jacoco/index.html

cd ../academicService
mvn clean test jacoco:report

cd ../attendanceService
mvn clean test jacoco:report
```

```
cd ../apiGetaway
mvn clean test
```

3.2 Todos los backends (desde la raíz del workspace)

```
foreach ($s in @("authService","academicService","attendanceService","apiGetaway")) {
    Push-Location $s
    mvn clean test jacoco:report -q
    Pop-Location
}
```

3.3 Ubicación de reportes

Servicio	Reporte JaCoCo HTML	Reportes Surefire
authService	authService/target/site/jacoco/index.html	authService/target/surefire-reports/
academicService	academicService/target/site/jacoco/index.html	academicService/target/surefire-reports/
attendanceService	attendanceService/target/site/jacoco/index.html	attendanceService/target/surefire-reports/

Copias para el encargo: informe-ep3/jacoco-reports/{servicio}/index.html

4. Métricas de cobertura por servicio

4.1 authService — 91 % instrucciones

Paquete	Instrucciones	Ramas
controller	97 %	75 %
service.impl	90 %	71 %
util	82 %	50 %
exception	100 %	n/a

Clases de test (7):

- AuthControllerTest — login, perfil, errores HTTP
- AdminUserControllerTest — CRUD usuarios admin
- AuthServiceImplTest — lógica login/registro
- AdminUserServiceImplTest — provisión de usuarios

- `JwtFilterTest` — filtro de seguridad
- `JwtUtilTest` — generación y validación JWT
- `GlobalExceptionHandlerTest` — respuestas de error estandarizadas

4.2 academicService — 71 % instrucciones

Paquete	Instrucciones	Ramas
<code>exception</code>	98 %	72 %
<code>util</code>	94 %	82 %
<code>security</code>	76 %	59 %
<code>controller</code>	73 %	45 %
<code>service</code>	69 %	47 %
<code>validation</code>	64 %	34 %
<code>service.impl</code>	58 %	37 %

Clases de test (27): cubren controllers (estudiantes, cursos, docentes, matrículas, evaluaciones, notas, apoderados), servicios, validadores, seguridad JWT y manejo de excepciones.

4.3 attendanceService — 80 % instrucciones

Paquete	Instrucciones	Ramas
<code>validation</code>	100 %	94 %
<code>exception</code>	100 %	n/a
<code>service</code>	94 %	66 %
<code>security</code>	82 %	76 %
<code>util</code>	80 %	33 %
<code>controller</code>	74 %	40 %
<code>service.impl</code>	73 %	35 %

Clases de test (13): sesiones, asistencia, anotaciones, servicios, validación, JWT y excepciones.

4.4 apiGetaway — smoke test

- `ApiGetawayApplicationTests.contextLoads` — verifica que el contexto Spring Boot arranca correctamente con la configuración de rutas del gateway.

5. Gráfico de cobertura (instrucciones)

authService		91%
attendanceService		80%
academicService		71%
apiGetaway	(smoke test – sin métricas JaCoCo)	

6. Ejemplos de pruebas realizadas

6.1 authService — Login exitoso (`AuthServiceImplTest`)

```
@Test
void login_success() {
    LoginRequest request = new LoginRequest();
    request.setUsername("prof_castillo");
    request.setPassword("test1234");

    when(userRepository.findByUsername("prof_castillo")).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("test1234", "encoded")).thenReturn(true);
    when(jwtUtil.generateToken(user)).thenReturn("mock-jwt-token");

    AuthResponse response = authService.login(request);

    assertNotNull(response.getAccessToken());
    assertEquals("mock-jwt-token", response.getAccessToken());
    verify(userRepository).findByUsername("prof_castillo");
}
```

Resultado: Tests run: 4, Failures: 0, Errors: 0 (clase `AuthControllerTest`)

6.2 academicService — Controller con `MockMvc`

Las pruebas de controllers usan `@WebMvcTest` + `MockMvc` para simular peticiones HTTP sin levantar la BD:

```
@WebMvcTest(StudentController.class)
class StudentControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean StudentService studentService;
    @MockBean JwtFilter jwtFilter;

    @Test
    @WithMockUser(roles = "ADMIN")
    void listStudents_returns200() throws Exception {
        when(studentService.findAll()).thenReturn(List.of());
    }
}
```

```

        mockMvc.perform(get("/students"))
                .andExpect(status().isOk());
    }
}

```

6.3 attendanceService — Validación de negocio

```

@Test
void createAttendance_invalidStudent_throwsException() {
    when(studentValidator.exists(999L)).thenReturn(false);
    assertThrows(ResourceNotFoundException.class,
        () -> attendanceService.create(dtoWithStudent(999L)));
}

```

6.4 Resultado Surefire (extracto real)

```

Test set: cl.duoc.libroDigital.authService.controller.AuthControllerTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.238 s

```

7. Organización del código de pruebas

```

{servicio}/src/test/java/
└─ cl/duoc/libroDigital/{servicio}/
    ├── controller/      ← @WebMvcTest, MockMvc
    ├── service/         ← @ExtendWith(MockitoExtension.class)
    ├── security/        ← JwtFilter, SecurityConfig
    ├── util/            ← JwtUtil, helpers
    ├── validation/      ← reglas de negocio
    └─ exception/        ← GlobalExceptionHandler

```

Convenciones:

- Nombre de clase: {ClaseBajoPrueba}Test
- Métodos: {acción}_{escenario} (ej. login_success, login_invalidPassword)
- Sin dependencia de PostgreSQL en tests unitarios (repositorios mockeados)

8. Componente frontend (frontend-react)

8.1 Herramientas

Herramienta	Uso
Vitest 4	Runner de pruebas (integrado con Vite)
React Testing Library	Renderizado y interacción en componentes
@vitest/coverage-v8	Métricas de cobertura (reporte HTML)
jsdom	Entorno DOM para tests sin navegador

8.2 Ejecución

```
cd frontend-react
npm install
npm test           # ejecutar una vez
npm run test:watch  # modo interactivo
npm run test:coverage # genera reporte en coverage/index.html
```

8.3 Resultados (2026-06-20)

Métrica	Valor
Archivos de test	10
Tests ejecutados	53
Fallos	0
Cobertura global (líneas)	6,22 %
Cobertura global (ramas)	3,49 %

La cobertura global es baja porque las pantallas CRUD (`Students.tsx` , `Grades.tsx` , etc.) tienen miles de líneas y se validan manualmente. La estrategia prioriza **lógica crítica desacoplada**: autenticación, permisos RBAC y validadores de formulario.

8.4 Cobertura por módulo crítico

Archivo / paquete	Líneas	Ramas	Qué se prueba
auth/permissions.ts	94 %	88 %	RBAC por rol, módulos, permisos CRUD
auth/AuthContext.tsx	86 %	64 %	Sesión, login, logout, localStorage
Login.tsx	92 %	64 %	Validación, login exitoso, errores API
api/client.ts	100 %	83 %	URLs, headers JWT

Archivo / paquete	Líneas	Ramas	Qué se prueba
utils/formatRut.ts	97 %	68 %	Validación RUT chileno
utils/validateEmail.ts	100 %	67 %	Formato email
utils/validatePhone.ts	93 %	85 %	Teléfonos chilenos
utils/validateDate.ts	95 %	85 %	Fechas, ponderaciones

8.5 Organización de tests

```
frontend-react/src/
├─ api/client.test.ts
├─ auth/
│   ├─ AuthContext.test.tsx
│   └─ permissions.test.ts
├─ Login.test.tsx
└─ utils/
    ├─ formatRut.test.ts
    ├─ formatStudentFullName.test.ts
    ├─ sortById.test.ts
    ├─ validateDate.test.ts
    ├─ validateEmail.test.ts
    └─ validatePhone.test.ts
```

8.6 Ejemplo — permisos RBAC (permissions.test.ts)

```
it("teacher cannot access admin-only modules", () => {
  expect(canAccessModule(["DOCENTE"], "guardians")).toBe(false);
  expect(canAccessModule(["DOCENTE"], "enrollments")).toBe(false);
});

it("admin can create in writable modules but not read-only ones", () => {
  expect(canCreateInModule(["ADMINISTRADOR"], "courses")).toBe(true);
  expect(canCreateInModule(["ADMINISTRADOR"], "attendance")).toBe(false);
});
```

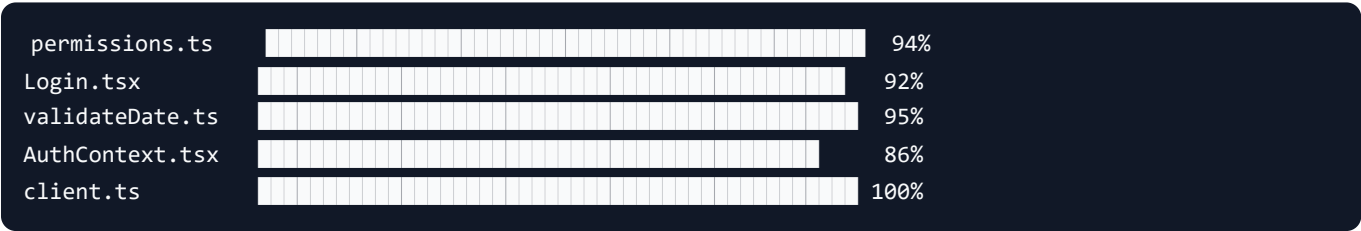
8.7 Ejemplo — Login (Login.test.tsx)

```
it("shows validation error when fields are empty", async () => {
  renderLogin();
  fireEvent.click(screen.getByRole("button", { name: "Ingresar" }));
  expect(await screen.findByText(/usuario y contraseña/i)).toBeInTheDocument();
});
```


8.8 Reporte de cobertura

- **HTML:** frontend-react/coverage/index.html (generar con npm run test:coverage)
- **Copia para encargo:** informe-ep3/coverage-frontend/index.html

8.9 Gráfico comparativo (módulos críticos frontend)



9. Pruebas de integración (complementarias)

Además de las unitarias, se dispone de:

Herramienta	Archivo	Alcance
Postman	infraestructura/postman/Libro_Digital.postman_collection.json	Flujo login → CRUD vía gateway :8090
Manual E2E	Frontend + 4 backends levantados	Flujos por rol en navegador

Ver guía: `infraestructura/postman/README.md`

10. Análisis y conclusiones

Fortalezas

- **318 tests** automatizados (265 backend + 53 frontend) con **0 fallos**
- Cobertura $\geq 71\%$ en los tres microservicios con lógica de negocio
- **authService** alcanza **91 %** — núcleo de seguridad bien cubierto
- **Frontend:** RBAC, login y validadores con **86–100 %** en módulos críticos
- Tests aislados (mocks) — ejecución rápida
- Reportes JaCoCo (backend) y Vitest HTML (frontend) incluidos en el encargo

Áreas de mejora

- Ramas (branches) en academicService (49 %) — ampliar casos edge en validadores
- `apiGetaway` — agregar tests de integración de rutas con `WebTestClient`
- Frontend — ampliar cobertura a pantallas CRUD con tests de integración (MSW + RTL)

Conclusión

Las pruebas unitarias garantizan la calidad de los componentes críticos (autenticación JWT, CRUD académico, registro de asistencia, permisos y validaciones del frontend) y proporcionan **métricas objetivas de cobertura** mediante JaCoCo y Vitest, cumpliendo los requisitos del informe de la Evaluación Parcial N°3.